

Neo4j Database Structure:

Single-Level Database Architecture:

text

Neo4j Instance

- └─ system (internal management)
- └─ neo4j (default)
- └─ database1
- └─ database2
- └─ database3

What you CAN do:

1. Multiple Databases (Neo4j 4.0+)

cypher

// Create separate databases

CREATE DATABASE music;

CREATE DATABASE users;

CREATE DATABASE analytics;

// Switch between them

:use music;

:use users;

2. Use Labels for "Virtual Databases"

cypher

// Use label prefixes as namespaces

CREATE (:music:Artist {name: "Artist1"});

CREATE (:music:Album {title: "Album1"});

CREATE (:users:User {name: "User1"});

CREATE (:users:Profile {email: "user@example.com"});

// Query by "namespace"

MATCH (n:music) RETURN n;

MATCH (n:users) RETURN n;

3. Use Properties for Categorization

cypher

CREATE (:Entity {db: 'music', type: 'artist', name: 'Artist1'});

CREATE (:Entity {db: 'music', type: 'album', title: 'Album1'});

CREATE (:Entity {db: 'users', type: 'user', name: 'User1'});

// Query by "database"

MATCH (n {db: 'music'}) RETURN n;

Alternative Approaches:

A. Multiple Neo4j Instances

yaml

docker-compose.yml

services:

neo4j-music:

image: neo4j:latest

ports: ["7474:7474", "7687:7687"]

environment:

NEO4J_AUTH: neo4j/password

neo4j-users:

image: neo4j:latest

```
ports: ["7475:7474", "7688:7687"]
```

```
environment:
```

```
  NEO4J_AUTH: neo4j/password
```

B. Single Database with Clear Separation

```
cypher
```

```
// Use consistent labeling conventions
```

```
CREATE (:Music:Artist {id: 1, name: "Taylor Swift"});
```

```
CREATE (:Music:Album {id: 1, title: "1989"});
```

```
CREATE (:App:User {id: 1, username: "john_doe"});
```

```
CREATE (:App:Session {id: 1, token: "abc123"});
```

Why No Nested Databases:

- **Graph nature:** Relationships can cross any boundary
- **Performance:** Single-level structure is more efficient
- **Simplicity:** Easier to query and maintain

Recommended Approach:

Use **labels as namespaces** for logical separation within a single database. This gives you the organizational benefits without the complexity of true nested databases.

Bottom line: Neo4j uses a flat database structure - you have multiple databases at the same level, but cannot nest them hierarchically.

.wslconfig

Step 1: Check current .wslconfig location and content

From **Windows PowerShell** (**not Docker Desktop**):

```
powershell
```

```
# Check if file exists
```

```
Test-Path "$HOME\.wslconfig"
```

```
# View file content
```

```
Get-Content "$HOME\.wslconfig"
```

```
# Check file encoding
```

```
Get-Item "$HOME\.wslconfig" | Format-List Name, Length, Encoding
```

Step 2: Recreate the file properly

```
powershell
```

```
# Delete existing file
```

```
Remove-Item "$HOME\.wslconfig" -ErrorAction SilentlyContinue
```

```
# Create with correct encoding
@"
[wsl2]
memory=8GB
processors=4
swap=2GB
localhostForwarding=true
"@ | Out-File -FilePath "$HOME\.wslconfig" -Encoding ASCII -NoNewline

# Verify
Get-Content "$HOME\.wslconfig"
```

Step 3: Force WSL to reload configuration

```
powershell
# Shutdown WSL completely
wsl --shutdown
```

```
# Wait 1 minute
Start-Sleep -Seconds 60
```

```
# Check WSL status (should return nothing)
wsl --list --verbose
```

```
# Restart Docker Desktop manually
```

Step 4: Verify the memory increase worked

After Docker Desktop restarts, check from **Windows PowerShell**:

```
powershell
# Start a WSL session to check memory
wsl -d Ubuntu free -h
You should see ~8GB total memory now.
```

If memory is still low (1.2GB), try this **nuclear option:**
Create the .wslconfig in the WSL distribution directory:

```
powershell
# Also create in WSL distribution config
@"
[wsl2]
memory=8GB
processors=4
swap=2GB
localhostForwarding=true
"@ | Out-File -FilePath
"C:\Users\jay_s\AppData\Local\Packages\CanonicalGroupLimited.Ubuntu_79rhkp1fndgsc\LocalState\.wslco
nfig" -Encoding ASCII
Then shutdown and restart again.
```

Now restart your Neo4j import:

Step 1: Clean up and start fresh
powershell

```
cd C:\Users\jay_s\book\docker
```

```
# Clean up previous failed attempts
```

```
docker compose down
```

```
Remove-Item -Recurse -Force data -ErrorAction SilentlyContinue
```

```
New-Item -ItemType Directory -Force -Path data
```

Step 2: Run the import with adequate memory

```
powershell
```

```
docker compose up neo4j-import
```

Step 2: Run the import with adequate memory

```
powershell
```

```
docker compose up neo4j-import
```

```
# In a separate PowerShell window, check progress
```

```
docker logs neo4j-import --tail 5
```

```
# Check for any new log entries
```

```
docker logs neo4j-import --since 5m
```

If that does not work, you can try small data set files:

Option 1: Use a much smaller dataset first (Recommended)

```
powershell
```

```
# Stop the current import
```

```
docker compose down
```

```
# Create tiny test files (first 1000 rows only)
```

```
Get-Content import/full/artist.csv -Head 1001 | Out-File import/artist_tiny.csv -Encoding UTF8
```

```
Get-Content import/full/album.csv -Head 1001 | Out-File import/album_tiny.csv -Encoding UTF8
```

```
Get-Content import/full/track.csv -Head 1001 | Out-File import/track_tiny.csv -Encoding UTF8
```

```
Get-Content import/full/playlist.csv -Head 1001 | Out-File import/playlist_tiny.csv -Encoding UTF8
```

```
# Clean data directory
```

```
Remove-Item -Recurse -Force data
```

```
New-Item -ItemType Directory -Force -Path data
```

```
# Test with tiny dataset
```

```
docker run --rm `
```

```
-v "${PWD}/import:/import" `
```

```
-v "${PWD}/data:/data" `
```

```
neo4j:5.26.0 `
```

```
/var/lib/neo4j/bin/neo4j-admin database import full book `
```

```
--nodes=:Track=/import/track_tiny.csv `
```

```
--nodes=:Playlist=/import/playlist_tiny.csv `
```

```
--nodes=:Artist=/import/artist_tiny.csv `
```

```
--nodes=:Album=/import/album_tiny.csv `
```

```
--skip-duplicate-nodes --skip-bad-relationships=true --multiline-fields=true --verbose
```

Option 2: Check for CSV file corruption

powershell

Check file sizes - they should be decreasing as import progresses

Get-ChildItem import/full/*.csv | Select-Object Name, Length

Check if files are accessible

docker run --rm -v "\${PWD}/import:/import" neo4j:5.26.0 ls -la /import/full/ | head -10

Option 3: Try a different Neo4j version

powershell

Use an older, more stable version

docker run --rm `

-v "\${PWD}/import:/import" `

-v "\${PWD}/data:/data" `

neo4j:5.19.0 ` *# Older stable version*

/var/lib/neo4j/bin/neo4j-admin database import full book `

--nodes=:Track=/import/full/track_headers.csv,/import/full/track.csv `

... rest of your command

To completely wipe out a Neo4j database and remove all labels, nodes, relationships, constraints, and indexes, use these commands:

Method 1: Delete Everything (Recommended)

cypher

// Delete all nodes and relationships

MATCH (n)

DETACH DELETE n;

// Drop all constraints

CALL apoc.schema.assert({}, {});

// Or manually drop constraints if above doesn't work

SHOW CONSTRAINTS

YIELD name

CALL {

WITH name

DROP CONSTRAINT name

} IN TRANSACTIONS;

```
// Drop all indexes
SHOW INDEXES
YIELD name
WHERE name IS NOT NULL
CALL {
  WITH name
  DROP INDEX name
} IN TRANSACTIONS;
```

Method 2: Using APOC (More Thorough)

```
cypher
// Install APOC if not already available
CALL apoc.help('apoc') yield name LIMIT 1;
```

```
// Delete everything including schema
CALL apoc.periodic.iterate(
  "MATCH (n) RETURN n",
  "DETACH DELETE n",
  {batchSize: 10000}
);
```

```
// Reset entire database schema
CALL apoc.schema.assert({}, {});
```

Method 3: Nuclear Option - Delete Database Files

If the database is corrupted or you want a completely fresh start:

Stop Neo4j first:

```
powershell
docker compose down
```

Delete database files:

```
powershell
Remove-Item -Recurse -Force data -ErrorAction SilentlyContinue
New-Item -ItemType Directory -Force -Path data
```

Restart Neo4j:

```
powershell
docker compose up -d
```

Method 4: Verification Queries

After cleaning, verify everything is gone:

```
cypher
// Check if any nodes remain
MATCH (n) RETURN count(n) AS remaining_nodes;
```

```
// Check constraints
SHOW CONSTRAINTS;
```

```
// Check indexes  
SHOW INDEXES;
```

```
// Check labels (should return empty)  
CALL db.labels();
```

```
// Check relationship types (should return empty)  
CALL db.relationshipTypes();
```

Method 5: For Specific Database (Multi-database setup)

If you have multiple databases:

```
cypher
```

```
// Switch to database you want to wipe
```

```
:use system
```

```
// Create or replace the database (complete wipe)  
CREATE OR REPLACE DATABASE book;
```

```
// Switch back to your database  
:use book
```

Recommended Approach:

Use **Method 1** if Neo4j is running and accessible. Use **Method 3** if the database is corrupted or you're having import issues.

For your current situation, since you're having import problems, I recommend **Method 3** (delete files and start fresh) as it's the most reliable for troubleshooting.

The primary difference between these two Neo4j procedures lies in their **accuracy** and **performance**. While both visualize the graph schema, they use different underlying methods to gather data.

1. Accuracy and "False Positives"

- **db.schema.visualization()**: Relies solely on database statistics. Because these statistics only track relationship types and node labels separately, this procedure can report **false positives**—showing relationships between labels that do not actually exist in your data.
- **apoc.meta.graph()**: Is much more accurate because it **traverses the actual graph** to confirm relationships. It post-processes results to remove any non-existent relationships identified by statistics.

2. Performance and Resource Use

- `db.schema.visualization()`: Extremely fast, even for massive datasets, because it only reads metadata statistics rather than scanning individual records.
- `apoc.meta.graph()`: Slower and more resource-intensive as it must scan nodes and relationships to verify their existence. On very large graphs, this can take a significant amount of time unless [sampling parameters](#) are used to limit the scan.

3. Features and Outputs

- `db.schema.visualization()`: A built-in Neo4j procedure that provides a simple visual overview of labels, types, indexes, and constraints.
- `apoc.meta.graph()`: Part of the APOC library. It returns a **virtual graph** which can include additional metadata such as node counts and relationship degrees. It also offers advanced variations like [apoc.meta.graph.of](#) to visualize the schema of a specific sub-graph query.

Summary Comparison Table

Feature	<code>db.schema.visualization()</code>	<code>apoc.meta.graph()</code>
Accuracy	Lower (possible false positives)	Higher (validates against data)
Speed	Very Fast (uses statistics)	Slower (scans graph data)
Dependency	Built-in	Requires APOC Plugin
Best For	Quick blueprint overview	Accurate schema validation